Interfaces or Bodies? A Preregistered Test of ε-Sufficient Context for Language-Model Code Editing in Go

Jude Castiel Johansen
Shockware Limited, independent researcher
castiel@shockware.co.uk ORCID 0009-0002-5644-3073

Abstract—Coding agents decide, many times a day, how much of a repository to show a language model, and the decision is made by heuristic or by end-task scores that confound localisation, context size, and model quality. A recent note defines ε-sufficient context: a context is sufficient for an editor when the editor's success on it is within ε of its success on the best context in a stated family, and proves that under a bounded reader this property is not closed under superset. We propose a preregistered, paired, multi-model study that operationalises the definition as a non-inferiority test. Holding localisation fixed at the gold write-set on at least 300 post-cutoff fail-to-pass instances mined from five to eight Go repositories, we compare (H2) a compiler-computed context consisting of the write-set and the exported interfaces of its dependencies against the same package set with full bodies and against the transitive slice; (H1) we estimate each model's degradation curve by padding a fully covered base context to fixed absolute loads up to 200k tokens with three padding types; (H3) we measure, descriptively, whether showing the forward slice reduces breakage of passing tests; and (H4) we report tokens per arm. Coverage is defined operationally per arm by checking that the gold patch is expressible from the context. Three models, temperature zero with three samples per cell on the primary arms, paired analysis with Wilson score intervals and Holm correction. The instances, slices, and contexts are released as SuffBench v0.1. Nothing has been run; a 30-instance pilot, disjoint from the confirmatory set, will fix the non-inferiority margin from the measured run-to-run flip rate.

I. Introduction

Every system that uses a language model to edit code must choose which parts of the repository to place in the model's context. The choice is consequential in two directions. Too little, and the model lacks a fact it needs; too much, and its accuracy falls with input length even when the needed facts are present [1], [2], [3]. Current practice chooses by retrieval heuristics or by the agent's own exploration [4], [5], and evaluates the choice only through the end-task pass rate, which confounds whether the right place was found, how much was shown, and how good the model is.

A recent definitions note [6] separates these. A change request is a contract (Δ, D, I, W): requested behaviour, scope of permitted behavioural change, invariants, and a write-set. A context K is ε-sufficient for an editor A when A's success on K, judged by an oracle of tests, is within ε of its success on the best context in a stated candidate family. Under a stylised model in which success factors into coverage of needed facts and a non-increasing function of load, the note proves, with a Lean 4 mechanisation, that sufficiency is not closed under superset: adding irrelevant material to a sufficient context can make it insufficient, and the admissible tolerance grows with the amount added. The note also states three properties a language could enforce so that the sufficient context becomes a graph query: dependencies matter only through their declared interfaces (P1); dependents rely only on what interfaces pin down (P2); and the context is computable from the dependency graph before the edit (P3). A mutation study on one Go repository found that interface-preserving faults were caught by tests outside the changed package in about one case in twenty-three, concentrated in shared helper packages.

None of that is evidence about real editors. This report proposes the study that would supply it, designed to be cheap enough for an independent researcher and powered enough to be worth running. Its central question is P1 as an experiment: does an editor given only the interfaces of what the write-set depends on do as well as one given the bodies, and as well as one given the transitive slice? Framed as non-inferiority, this is exactly the ε-sufficiency definition. Around it we estimate the degradation curve per model at verified fixed coverage, measure the forward slice's effect on regressions descriptively, and report the token cost of every arm.

A. Relation to recent work

Several 2026 measurements bear directly on the design and we build on them rather than claim to precede them. Sam-Bodden [7] holds localisation fixed on SWE-bench Verified and varies the representation of surrounding Python code, finding that signatures and skeletons serve about as well as source; his preregistered superiority test failed to find a difference, which is precisely the gap a powered non-inferiority test fills. Mohammadi et al. [8] show across seven models that availability of needed facts decides success and their position does not. Du et al. [3] show that length alone hurts even under perfect retrieval and neutral padding. Xue [9] reports large drops from short to long contexts on two auditing tasks. Alonso et

al. [10] reduce agent regressions on SWE-bench Verified from about six percent to under two with a graph-based impact map. Jia, Barr and Mechtaev [11] search for minimal sufficient context on the editor side. Hierarchical context pruning [12] found that pruning function bodies in dependent files did not significantly reduce completion accuracy. Bjarnason et al. [13] quantify run-to-run randomness in agent evaluation, which bounds what any paired comparison can resolve and motivates our pilot.

What remains open, and what this study measures: the interfaces-versus-bodies question as a powered non-inferiority test in a statically typed language with type-graph-derived interfaces; a per-model degradation curve on repository editing with verified coverage and intervals; the forward slice's effect in a single-shot setting stratified by whether the write-set is a shared helper; and a dataset whose relevance labels come from test verdicts rather than from workflow signals or gold-patch files [14], [15].

II. Hypotheses and estimands

All contrasts are paired by instance. p_X denotes the population-average probability, over instances and samples, that a patch produced from context X passes the fail-to-pass tests (Δ-pass). Where stated, the outcome is instead the conjunction Δ-pass and frame-pass (all pass-to-pass tests still pass).

- H2 (primary): interface sufficiency. $p_{\mathrm{A1}} \geq p_{\mathrm{A1b}} - \varepsilon$ and $p_{\mathrm{A1}} \geq p_{\mathrm{A2}} - \varepsilon$, where A1 is write-set bodies plus interfaces of direct dependencies, A1b the same package set with bodies, and A2 the transitive slice. Non-inferiority margin ε fixed after the pilot (Section V); primary candidate $\varepsilon = 0.075$, secondary $\varepsilon = 0.05$. Estimand: population-average over instances. Per-instance ε-sufficiency is estimated only on the subset with three samples per cell.
- H1: degradation. At fixed, verified coverage, Δ-pass is non-increasing in absolute load. Tested as A1 versus A1 padded to 128k and to 200k tokens (superiority, one-sided), separately for three padding types; the full curve at {base, 32k, 64k, 128k, 200k} is reported with intervals.
- H3 (descriptive): forward slice. The rate of Δ-pass-and-frame-pass for A1F (A1 plus interfaces and test names of the forward slice) versus A1, overall and stratified by whether the write-set is a shared helper (number of dependents at or above the median). Reported with intervals; not a confirmatory test unless shared-helper oversampling yields at least 20 frame failures.
- H4: cost. Tokens, latency, and price per arm; the ratio of A1 to A2 and to lexical retrieval at matched and at large budgets.
- Control: parametric memory. $p_{\mathrm{A}\emptyset}$, task text only, estimates how often the model succeeds with no repository context; reported per repository.

III. Instances: SuffBench v0.1

Sources. Five to eight actively developed, single-module Go repositories with a green Linux baseline, selected before mining by stars, activity, and test density; the GitHub CLI will be one. The Multi-SWE-bench harness and container recipes [16] are reused.

Mining. For each commit after the latest documented training cutoff of any evaluated model, touching both non-test and test files: build at parent and child; the child's test files are applied to the parent; a test is fail-to-pass iff its package compiles at the parent with the child's tests and it fails there and passes at the child; the full suite must be green at the child. Instances whose fail-to-pass tests reference identifiers introduced by the gold patch are excluded unless the new signatures are supplied as specification, in which case the instance is flagged. The write-set is the set of changed packages; the interface write-set is computed by exported-API diff.

Task text. The linked issue body where one exists; otherwise a behaviour-only specification rewritten from the PR body by a language model with all identifiers and file names removed, hand-checked by the author. The source is recorded and results are reported by source.

Screening. Every instance is screened manually for under-specification and leakage, about ten minutes each, following SWE-bench Verified's protocol. Target: at least 300 confirmatory instances plus 30 disjoint pilot instances.

Slices and checks. For each instance: backward and forward slices from go list -deps -test; a coverage slice from per-package coverage of the verdict tests on the gold tree; a mutation slice using body-only, interface-preserving operators; a closure check that the write-set with its backward and forward slices covers the import closure of the executed test binaries; and, per arm, an expressibility check that every identifier, declaration, and import the gold patch references is present in the context. Instances where the gold patch adds imports outside the parent's dependency graph are flagged.

Flakiness. Every test that fails on any run is rerun three times on the clean tree; those failing on the clean tree are excluded and listed.

Release. Under embargo until Stage 1 submission; then on Zenodo with the datasheet required by the public SuffBench specification [17].

IV. Design

Arms. Table I. Localisation is fixed at the gold write-set W in all arms but the retrieval baselines. Contexts are serialised deterministically with W last, immediately before the task.

Editor protocol. Single shot: task text and context in, edits out in a search-and-replace block format, applied with a strict matcher. One format-repair turn is allowed only when a block fails to apply; no test feedback is ever given. Temperature zero. Three samples per cell on A1,

Arm	Content	Typical load
A∅	task text only	< 2k
A0	W bodies	3k
A1	W bodies + interfaces of deps(W)	15k
A1b	W bodies + bodies of deps(W)	40k
A1F	A1 + interfaces and test names of $F(W)$	25k
A2	W + bodies and tests of $B(W) \cup F(W)$	150k
A4s	lexical retrieval at A1's budget	15k
A4l	lexical retrieval at a large budget	150k
P_{32}..P_{200}	A1 padded to 32k, 64k, 128k, 200k	as named

TABLE I
Arms. Padding types: same-repository packages outside the test-binary closure with a lexical-similarity ceiling; cross-repository Go; whitespace. Loads are medians from the GitHub CLI; A2 and A4l run only on models with windows above 200k.

A1b, A2; one elsewhere. Provider and quantisation are pinned and recorded.

Models. One frontier model via its vendor API on A1, A1b, A2, P_{128}, P_{200}, and A∅ over a stratified random 100-instance subset; two open-weights models via a hosted API on all arms and all instances. Documented training cutoffs are recorded and every instance postdates them.

Verification. Stage one runs the fail-to-pass tests; stage two runs the full suite only for patches that pass stage one. Outcomes: applied, Δ-pass, frame-pass, tokens in and out, latency.

V. Analysis plan

Pilot. Thirty disjoint instances, one open-weights model, all arms, three samples per cell. It yields the run-to-run flip rate at temperature zero and the between-arm disagreement rate π_d. Pilot data are exploratory and are not reused.

Margin. ε is set so that it exceeds the pilot's run-to-run flip rate by a stated factor and so that the confirmatory design has at least 80% power at the pilot's π_d. Sensitivity: with $n = 300$ and one-sided $\alpha = 0.05$, power at a true difference of zero is 0.99 for $\varepsilon = 0.10$ and 0.90 for $\varepsilon = 0.075$ at $\pi_d = 0.2$, and 0.61 for $\varepsilon = 0.05$. Required n for 80% power is $6.18\,\pi_d/\varepsilon^2$: 124 at $\varepsilon = 0.10$, 220 at $\varepsilon = 0.075$, 494 at $\varepsilon = 0.05$, all at $\pi_d = 0.2$. If the pilot shows $\pi_d > 0.3$, the primary margin becomes 0.10 and this is stated before unblinding.

Tests. H2 uses an intersection–union non-inferiority test: both comparisons must hold, each judged by the lower bound of a paired score interval for a difference of correlated proportions exceeding $-\varepsilon$. H1 uses one-sided paired tests of superiority with the same interval. Holm correction is applied across the family of confirmatory tests per model (two for H2, two per padding type for H1). Intervals on all proportions are Wilson intervals; bootstrap over instances is reported alongside.

No interim stopping. The sample size is fixed in advance.

Reporting. All arms, all models, all instances, in a public table; every pre-specified contrast whether or not significant; every deviation from this plan, with reasons, in a dedicated section.

VI. Threats and limitations

Single-shot editing is not an agent loop; the study measures context, not agency, and says so. Three models and one language. Gold localisation is a ceiling; the retrieval arms are the realistic floor. Task text may leak the fix despite screening; results are reported by source and the parametric-memory arm bounds the effect. Padding from the same repository can raise coverage; the similarity ceiling and the two other padding types address this. Temperature zero is not deterministic across providers [13]; the pilot measures the flip rate and the margin is set above it. Coverage is checked syntactically through expressibility; semantic needs beyond identifiers are not captured. The forward-slice question is likely under-powered and is labelled descriptive.

VII. Timeline and resources

Mining, screening, and slices: weeks 1 to 5 (about 50 hours of manual screening). Harness and pilot: weeks 6 and 7. Stage 1 submission: 20 November 2026. Confirmatory run after in-principle acceptance: three weeks. Analysis and Stage 2: by the EMSE deadline. Budget: model calls about $600 to $1,150 at current rates, verification on a laptop. All code, data, and prompts are released under Apache-2.0 and CC-BY-4.0.

AI-use disclosure

Anthropic's Claude (Fable 5.1) was used under the author's direction to draft this report, to write the analysis and tooling scripts, and to cross-check references; a separate model (Claude Opus) produced two rounds of critical review of the design, whose findings the author adopted or rejected. The author reviewed all text and takes full responsibility for it. Generative AI tools are not authors.

References

[1] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," Transactions of the Association for Computational Linguistics, vol. 12, pp. 157–173, 2024.

[2] M. Levy, A. Jacoby, and Y. Goldberg, "Same task, more tokens: The impact of input length on the reasoning performance of large language models," in Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL 2024), 2024, pp. 15 339–15 353.

[3] Y. Du, M. Tian, S. Ronanki, S. Rongali et al., "Context length alone hurts LLM performance despite perfect retrieval," arXiv:2510.05381, 2025.

[4] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang, "Demystifying LLM-based software engineering agents," Proceedings of the ACM on Software Engineering, vol. 2, no. FSE, pp. 801–824, 2025, describes the Agentless approach.

[5] R. Bairi, A. Sonwane, A. Kanade, V. D. C., A. Iyer, S. Parthasarathy, S. Rajamani, B. Ashok, and S. Shet, "CodePlan: Repository-level coding using LLMs and planning," Proceedings of the ACM on Software Engineering, vol. 1, no. FSE, pp. 675–698, 2024.

[6] J. C. Johansen, "ε-sufficient context for bounded readers: Definitions and a non-monotonicity result for code editing," Zenodo, version 1.1, 2026.
[7] B. Sam-Bodden, "What context does a coding agent actually need to act?" arXiv:2607.09691, 2026.
[8] B. Mohammadi, L. Klein, A. Chadha, A. Arora et al., "The working set of a coding agent: Coherence debt in repository-scale tasks," arXiv:2608.16630, 2026.
[9] Y. Xue, "When and how context rot appears in coding agents: A white-box study of agent skills in code auditing," arXiv:2607.17937, 2026.
[10] P. Alonso, S. Yovine, and V. A. Braberman, "TDAD: Test-driven agentic development: Reducing code regressions in AI coding agents via graph-based impact analysis," arXiv:2603.17973, 2026.
[11] H. Jia, E. T. Barr, and S. Mechtaev, "Compressing code context for LLM-based issue resolution," arXiv:2603.28119, 2026.
[12] L. Zhang, Y. Li, J. Li, X. Xia et al., "Hierarchical context pruning: Optimizing real-world code completion with repository-level pretrained code LLMs," arXiv:2406.18294, 2024.
[13] B. H. Bjarnason, A. Silva, and M. Monperrus, "On randomness in agentic evals," arXiv:2602.07150, 2026.
[14] B. Qin and Y. Xie, "Agent retrieval bench: Evaluating repository context retrieval for coding agents," arXiv:2607.24882, 2026.
[15] H. Li, L. Zhu, B. Zhang, R. Feng et al., "ContextBench: A benchmark for context retrieval in coding agents," arXiv:2602.05892, 2026.
[16] D. Zan, Z. Huang, W. Liu, H. Chen et al., "Multi-SWE-bench: A multilingual benchmark for issue resolving," arXiv:2504.02605, 2025.
[17] J. C. Johansen, "SuffBench: Specification for a context-sufficiency benchmark for code editing (v0.1.0-spec)," Shockware Limited, https://github.com/Shockware-Limited/suffbench, 2026.